

Ambiente de desenvolvimento

#UFV

Referências:

🔗 Esse documento foi elaborado tendo em vista o que foi desenvolvido ao longo da prova de conceito

Front-end:

🔗 JavaScript

- <https://www.w3schools.com/js/>
- <https://developer.mozilla.org/en-US/docs/Web/JavaScript>
- <https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference>
- <https://javascript.info/document>
- <https://youtu.be/Z7mnxUI4u00?si=KVPaVSut5TIUDzDn>

🔗 HTML

- <https://youtu.be/Fhy-5CtVkiM?si=NuVQDmn6aHhJbVHs>
- <https://developer.mozilla.org/pt-BR/docs/Web/HTML>
- https://www.w3schools.com/html/html_intro.asp

📄 CSS

- <https://youtu.be/AB35iSr1YyA?si=IbUTsO-X30rp1T22>
- <https://developer.mozilla.org/pt-BR/docs/Web/CSS>
- <https://www.w3schools.com/CSSref/index.php>

🔗 Existem jogos para o aprendizado de grid e flexbox

- [grid](#)

- [flexbox](#)

tailwind

- <https://tailwindcss.com/>
- <https://www.geeksforgeeks.org/css/tailwind-css/>
- <https://youtu.be/2RWsLmu8yVc?si=JpiQ1fhGjnI-Ny-n>

React

- <https://youtu.be/2RWsLmu8yVc?si=JpiQ1fhGjnI-Ny-n>
- <https://www.w3schools.com/REACT/DEFAULT.ASP>
- <https://pt-br.legacy.reactjs.org/tutorial/tutorial.html>
- <https://react.dev/learn/tutorial-tic-tac-toe>
- <https://vitest.dev/guide/>
- <https://betterstack.com/community/guides/testing/vitest-explained/>

Back-end:

Spring boot

- <https://spring.io/guides>
- <https://www.geeksforgeeks.org/advance-java/spring/>
- https://www.youtube.com/watch?v=n8_grrc8WN4
- https://symflower-com.translate.goog/en/company/blog/2024/how-to-do-mocking-spring-boot/?_x_tr_sl=en&_x_tr_tl=pt&_x_tr_hl=pt&_x_tr_pto=tc
- <https://docs.junit.org/current/user-guide/>
- <https://www.youtube.com/watch?v=6uSnF6luWlw>
- <https://medium.com/@iamfaisalkhatri/beginners-guide-to-junit-5-7756286cd4be>

Banco de dados:

SQL

- <https://www.w3schools.com/MySQL/default.asp>

🔗 JPA

- <https://spring.io/guides/gs/accessing-data-mysql>

Geral:

Aplicação *web* que utiliza o padrão arquitetural MVC, cuja stack usada é:

- **Infra e padronização do ambiente de desenvolvimento**
 - **Docker**
 - **.env** para aprimorar segurança da aplicação
 - **Makefile**
 - Facilitador do uso do ambiente
- **Back-end**
 - **Spring Boot**
 - Framework java amplamente usado no desenvolvimento *web*
 - API REST
 - Hibernate
 - Testes feitos com **JUNIT** e **JACOCO**
 - **Maven**
 - Gerenciador de dependências - facilita muito a integração de novos serviços para a aplicação
- **Front-end**
 - **React + javascript**
 - **Vite**
 - Linguagem/bibliotecas muito usadas para desenvolvimento *web* e com curva de aprendizado aceitável para o tempo de desenvolvimento da aplicação fornecido
 - Testes efetuados com **VITEST**
 - **Tailwind + css**
 - Estilização de páginas menos verbosa com curva de aprendizado aceitável
- **Banco de dados**
 - **MySQL**

📄 A organização dos arquivos da prova de conceito e da aplicação no geral, vai ser:

```
prova_conceito/    (ou desconecta/)
├─ backend/
├─ frontend/
├─ database/
├─ docker-compose.yaml
├─ .env
├─ Makefile
└─ ... (outros arquivos úteis)
```

Sobre as pastas:

- **backend** -> contempla todos os arquivos associados a testes java, código código, configuração do ambiente back-end, entre outros dessa natureza.
- **frontend** -> contemplam todos os arquivos associados ao desenvolvimento de telas, componentes, testes de componentes do front-end e, inicialmente, os assets utilizados pela aplicação, mas que podem ser movidos para a pasta database.
- **database** -> inicialmente vazia, pode-se inserir as consultas SQL dos júniores aqui. Mas, pode ser que aqui também entrem os assets usados no desenvolvimento.

Orientações:

 **Dúvidas ou sugestões? Chame o arquiteto!**

Será necessária a utilização de assets para o desenvolvimento das telas, para melhor organização desses elementos, estes devem ser colocados dentro da pasta `assets`.

Arquivos relacionados à Controle, Serviço, Repositório e Modelo devem ter uma pasta correspondente ao tipo de funcionalidade a ser implementada.

Exemplo de organização de arquivos:

```
provaConceito
├─ src
│   └─ main
│       └─ java
│           └─ com
│               └─ provaConceito
│                   └─ provaConceito
│                       ├── controle
│                       │   └─ controleContador.java
│                       ├── modelo
│                       │   └─ modeloContador.java
```

```
├── repositorio
│   └── repositorioContador.java
├── servico
│   └── servicoContador.java
└── ProvaConceitoApplication.java
```

Arquivos de componentes React devem ser organizados como se segue: na pasta `src/componentes` (componentes mais gerais); `src/componentes/modais` (componentes modais).

☰ Organização de componentes:

```
src/
├── App.css
├── App.jsx
├── componentes
│   ├── BotaoCliques.jsx
│   ├── BotaoCliquesMock.jsx
│   ├── modais
│   │   └── Modal.jsx
│   └── Parabens.jsx
├── index.css
└── main.jsx
```

Front-end:

O projeto foi construído usando o [vite](#), em suma, é um facilitador para desenvolvimento *web*.

🔗 Para adicionar novas dependências à aplicação, comunique o arquiteto de software

React

É uma biblioteca *javascript* para desenvolvimento front-end. O React se baseia em [SPA](#) (Single Page Application), ou seja, uma única página que vai se atualizando conforme as interações do usuário com a aplicação através de componentes e modais. Isso garante maior velocidade e dinamicidade para a aplicação.

🔗 De forma simples, o React é um javascript com HTML misturado nele.

☰ Na prova de conceito temos a seguinte organização dos arquivos para a pasta `src`:

```
> .
├── App.css
├── App.jsx
├── App.test.jsx
├── assets
│   ├── dadinho-pulando.png
│   ├── GIF nuvem.gif
│   └── react.svg
├── components
│   ├── BotaoCliques.jsx
│   ├── BotaoCliquesMock.jsx
│   ├── Modal.jsx
│   └── Parabens.jsx
├── index.css
├── main.jsx
└── setupTests.js
```

❗ A extensão usada pelo react é a **jsx**

❗ Usualmente, as funcionalidades usadas pela página são guardadas na pasta **components** e lá a organização vai de quem está codificando

Poderíamos ter a pasta `modal` dentro de `components` para guardar o arquivo `Modal.jsx`

⚠️ **Todo componente que for renderizado deve ter a primeira letra do arquivo correspondente maiúscula**

Ou seja, se quero renderizar o componente `BotaoCliques`, o seu nome de arquivo não poderia ser `botaoCliques.jsx`, mas sim `BotaoCliques.jsx`

🔗 Componentes e modais

De forma direta, **componentes** são as funcionalidades da aplicação. Eles são como o "back-end" do front-end. Os **modais** também são componentes, mas são dedicados ao UI que normalmente bloqueiam o acesso do usuário à tela principal, o pop-up é um exemplo de modal.

⚡ **Componentes devem retornar algo para serem aplicados na página web**

Artifícios muito úteis no react: **props** e **states**.

Props (properties) são um mecanismo para passar dados de um componente pai para um componente filho. Pense nelas como argumentos de uma função, mas para componentes. Elas são a principal forma de comunicação entre componentes no React, permitindo que você crie componentes reutilizáveis e dinâmicos.

States são literalmente estados do componente, eles renderizam o componente novamente (alterando seu estado). Essenciais para "reagir" às interações do usuário

☰ Exemplo de props e states

```
import {useState} from "react";

function App(){
  const [message, setMessage] = useState("Olá!"); //Primeiro elemento é a
  variável que terá o estado alterado e é inicializada com a função
  useState; Segundo elemento é a função que muda o estado da função

  //let message="Olá!"
  return(
    <div>
      <h1>{message}</h1> { /*props*/}
      <button onClick={() => {
        setMessage('Botão clicado!')} /*state*/}
      >Mudar mensagem</button>
    </div>
  )
}

export default App
```

📌 Características principais das Props

- **Fluxo de dados unidirecional:** Os dados fluem sempre de cima para baixo na árvore de componentes (do pai para o filho). Um componente filho não pode alterar as props que recebeu do pai.
- **Imutáveis:** Dentro de um componente filho, as props são somente leitura. Você não pode modificá-las diretamente. Se um componente precisa mudar seus próprios dados, ele deve usar um **"state"** (estado).
- **Podem ser qualquer tipo de dado:** Você pode passar strings, números, objetos, arrays, funções e até mesmo outros componentes como props.

```
// src/components/UserCard.js
import React from 'react';
```

```

function UserCard(props) {
  return (
    <div className="card">
      <h3>{props.name}</h3>
      <p>Idade: {props.age} anos</p>
    </div>
  );
}

export default UserCard;

// src/App.js
import React from 'react';
import UserCard from './components/UserCard';

function App() {
  return (
    <div>
      <h1>Usuários</h1>
      <UserCard name="Maria" age={28} />
      <UserCard name="João" age={35} />
      <UserCard name="Ana" age={22} />
    </div>
  );
}

export default App;

```

Funcionamento:

Uma página web é estruturada por páginas HTML e a página que serve como ponto de partida chama-se [index.html](#). No nosso caso, executamos um script dentro da página (existe tag própria para isso), para aplicar nossos arquivos React na página web. Algo que é muito falado na Internet quando o assunto é desenvolvimento web é que o HTML é o esqueleto, o CSS é a roupa e o Javascript é a personalidade. O que isso significa, o HTML é a estrutura (por exemplo, definir onde vai ser entrada de texto), o CSS é a estilização não afeta o funcionamento lógico da aplicação, apenas o visual e o javascript é a lógica por trás do funcionamento da aplicação. Dentro do escopo do PI, a atuação do javascript como agente pensante vai ser um pouco reduzida para dar espaço ao java. **Deixando mais claro, o javascript vai conversar com a classe controladora do back-end, essa classe controladora conversa com a classe de serviço que a partir do banco de dados, devolve um resultado para o front exibir. Em suma, a lógica feita pelo javascript/react é pedir coisas para o back-end e mudar o que é apresentado na tela.**

❗ Tags HTML são as formas do programador indicar o que cada elemento da tela vai ser. Existem inúmeras tags HTML


```
<h1>Maior tamanho de título</div>
<div>
  <p>Parágrafo dentro de uma tag "bloco de código"</p>
</div>
```

Algumas tags não precisam ser fechadas, pode-se quebrar linha sempre que quiser quando estiver usando tags

📁 As tags HTML formam a DOM (Document Object Model) Tree

O javascript pode explorar essa árvore e aplicar comportamentos específicos sobre os nós dessa árvore

```
<!DOCTYPE html>
<html>
<head>
  <title>Exemplo document.getElementById</title>
</head>
<body>

  <p id="meuParagrafo">Este é o texto original do parágrafo.</p>

  <button onclick="mudarTexto()">Clique aqui para mudar o texto</button>

  <script src="script.js"></script>

</body>
</html>
```

```
function mudarTexto() {
  // 1. Encontra o elemento com o ID 'meuParagrafo'
  const paragrafo = document.getElementById('meuParagrafo');

  // 2. Acessa a propriedade innerHTML para mudar o texto
  paragrafo.innerHTML = 'O texto do parágrafo foi alterado com sucesso!';

  // Você também pode mudar outras propriedades, como o estilo
  paragrafo.style.color = 'blue';
  paragrafo.style.fontWeight = 'bold';
}
```

Essa conversa é feita através de métodos HTTP e promises. Os métodos HTTP mais comuns são: *GET* (Pega do banco), *PUT* (Atualiza o banco), *POST* (Insere no banco) e

DELETE (Deleta do banco).

- ❶ Promises fazem o comportamento assíncrono da aplicação, funcionam literalmente como uma promessa. A promessa só é cumprida se o combinado for realizado corretamente

```
// Buscar o total inicial de cliques no backend
const fetchCliques = async () => {
  try {
    const res = await fetch(`${API_URL}/status`);
    const data = await res.json();
    setTotalCliques(data.total);
    setShowExtraButton(data.dezCliques);
  } catch (err) {
    console.error("Erro ao buscar total de cliques:", err);
  }
};

useEffect(() => {
  fetchCliques();
}, []);
```

- ❷ Explicando o código acima:

Temos uma constante que recebe uma função assíncrona (uma Promise, mas com sintaxe mais limpa e que sempre retorna uma promise) Try-catch onde o try pega da API o método status e produz o json que será usado em useEffect (*hook react - função pronta que lida com operações como dar fetch de dados*). Catch só imprime que deu erro (promessa não pôde ser cumprida)

Depois dessas explicações, basta chamar esses componentes no main.jsx que eles serão aplicados na página.

🔗 Usualmente, temos 2 arquivos principais: App.jsx e main.jsx

App.jsx pega todos os componentes e os entrega para main.jsx, mas isso não é obrigatório

Tailwind css

É um framework para facilitar o uso do css. O tailwind faz uso de estilização inline através de classes. Existem trilhões de propriedade do tailwind que não cabem explicar aqui, é

realmente ir atrás conforme a necessidade. Abaixo tem um exemplo de uso do tailwind no react. Onde há className é tailwind.

```
return (
  <div
    className="flex flex-col justify-center items-center min-h-screen
gap-5 p-5 bg-contain bg-center bg-fixed bg-slate-400"
    style={{ backgroundImage: `url(${funcao})` }}
  >
    <h1 className="text-2xl font-bold text-slate-900">
      Total de Cliques: {totalCliques}
    </h1>

    <button
      onClick={registrarClique}
      className="px-6 py-3 bg-blue-600 text-white rounded-2xl shadow-lg
hover:bg-blue-700 transition"
    >
      Clicar
    </button>

    {showExtraButton && (
      <button
        onClick={handleExtraClick}
        className="px-6 py-3 bg-green-500 text-white rounded-2xl shadow-
lg hover:bg-green-600 transition"
      >
        Mostrar Pop-up
      </button>
    )}

    {modalOpen && <Modal />}
  </div>
);
}
```

É importante citar alguns itens de css, como: *margin, padding, border*. São os controladores de espaço dos elementos da página. Margin controla o espaçamento externo dos elementos (border para elementos adjacentes); padding controla espaçamento interno dos elementos (border para elementos internos) e border é a fronteira entre o espaço interno e externo. E cada elemento tem seu tamanho próprio.

Back-end:

O back-end é constituído puramente pelo spring boot, tendo as dependências e pacotes gerenciadas pelo maven. O spring fornece uma gama de funcionalidades implementadas

em *notations* . Essas notations podem fazer consultas ao banco de dados, realizar a integração web, simplificar métodos, entre muitas outras funcionalidades.

☰ Exemplo de notation

As notations usadas aqui são utilizadas principalmente para realização de consultas no banco de dados. Por exemplo, @Entity e @Table são análogas a CREATE TABLE tabelaComNomeDaClasse

```
package com.provaConceito.provaConceito.modelo;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.Table;
import lombok.AllArgsConstructor;
import lombok.NoArgsConstructor;

@Entity
@Table(name = "modelo_contador")
@NoArgsConstructor
@AllArgsConstructor
public class modeloContador {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    public Long getId() {
        return this.id;
    }

    public void setId(Long id) {
        this.id = id;
    }

    @Column(nullable = false)
    private Long totalCliques;

    public Long getTotalCliques() {
        return this.totalCliques;
    }

    public void setTotalCliques(Long totalCliques) {
        this.totalCliques = totalCliques;
    }
}
```

```
}  
}
```

🔗 Para adicionar novas dependências à aplicação, comunique o arquiteto de software

REST, Hibernate e API

Essas são algumas palavras que aparecem muito entre canais dev no YouTube, mas nem sempre fica claro o que cada um desses termos significa. Vamos tentar destrinchar cada um.

Começando pelo o que eu acredito ser o mais famoso, **REST** é um modelo de arquitetura para sistemas web, é basicamente uma forma de como organizar nossa aplicação web. Existem outros modelos como: camadas, micro-serviços, eventos, etc. REST é uma sigla para REpresentational State Transfer, este modelo se baseia fortemente nos métodos HTTP (POST, GET, DELETE, PUT) e foca na manipulação de recursos (**na prova de conceito, foi usada a quantidade de cliques do usuário**), utilizando uma rota específica para cada forma de manipulação desse recurso. Esses métodos HTTP são responsáveis pela realização do CRUD (Create Read Update Delete) da aplicação.

Partindo da ideia do REST, temos o desenvolvimento de **APIs RESTful**, em outras palavras, APIs que obedecem a ideia do REST. Uma API (Application Programming Interface) é uma interface para interação do usuário com a lógica da aplicação que está sendo usada. Uma explicação muito comum para APIs é imaginar a API como um garçom que recebe o pedido do cliente (requisição do front) para a cozinha (back que atende a requisição do front) e entrega o pedido de volta ao cliente (resposta da requisição pelo back-end). **No nosso caso, nossa API seria a classe com @RestController.**

☰ Exemplo de uso de API

Uma aplicação que informa a condição meteorológica de um local consome uma API de meteorologia para fornecer a resposta para o usuário.

Hibernate é um **framework de Mapeamento Objeto-Relacional (ORM)** para a linguagem Java. Sua principal função é eliminar a necessidade de escrever SQL manualmente, mapeando classes Java diretamente para tabelas em um banco de dados relacional. Ele atua como uma **camada de abstração** entre a aplicação Java e o banco de dados. Com o Hibernate, você trabalha com **objetos Java** (entidades) em vez de registros de banco de dados, o que torna o código mais limpo e orientado a objetos. O Hibernate se encarrega de gerar e executar as consultas SQL por baixo dos panos. No ecossistema Spring, a integração com o Hibernate é feita por meio do **Spring Data JPA**, que simplifica ainda mais a persistência de dados.

Docker:

Gerenciar os compose, volumes, Dockerfiles, entre outras especificidades do Docker são responsabilidade do arquiteto

⚡ Dúvidas em relação ao uso do Docker devem ser passadas para o arquiteto poder tratá-las

📌 .env será repassado pelo arquiteto aos outros devs, caso haja necessidade real

Contudo, é importante saber alguns comandos para usar o ambiente de forma plena. Alguns comandos foram simplificados para acelerar a codificação em teste, mas nem sempre é possível cobrir todos cenários possíveis ao longo do ciclo de desenvolvimento.

Makefile:

Aqui estão a grande maioria dos comandos usados ao longo do ciclo de desenvolvimento da aplicação.

🔑 Comandos mais importantes:

- make up:
 - Inicia o compose -> substituto para `docker compose up`
- make down:
 - "Desliga" o compose e preserva os mounts (itens a serem persistidos dentro do container nesse caso) -> substituto para `docker compose down`
- make npm-list:
 - Confere os node modules do container frontend -> substituto para `docker compose exec -it frontend npm list`
- make spring-boot-run:
 - Executa o back-end -> substituto para `docker compose exec -it backend mvn spring-boot:run`
- make maven-dependencies-list:
 - Exibe as dependências do back-end em formato de lista -> substituto para `docker compose exec -it backend mvn dependency:list`
- make maven-dependencies-tree:
 - Similar ao make maven-dependencies-list, mas em formato de árvore -> substituto para `docker compose exec -it mvn dependency:tree`

⚠ **Dúvidas ou adição de novos comandos, avise o arquiteto!**

🔗 **Para acessar o banco remoto:**

`docker run -it --rm mysql:8.1 mysql -h (IP BANCO) -u (USUÁRIO) -p`

NÃO COLOQUE () NO COMANDO

Testes:

Temos dois tipos de teste: **cobertura** e **unitários**. Os testes e cobertura se baseiam no que foi executado ao longo dos testes, ou seja, conferem se o código entrou nos laços, quais linhas foram executadas, **não afirmam nada sobre resultados corretos ou incorretos dos testes**. Já os testes unitários realizam testes sobre classes buscando testar os métodos da classe, **não apresentam o que foi executado, apenas aprovam ou reprovam resultados**.

🔗 **Os logs de teste ficam na pasta target dentro do back-end (se teste de back-end) ou são exibidos no terminal (front-end e back-end)**

Os formatos de arquivo de log podem ser .txt, .xml ou mesmo .html. As subpastas para conferir esses logs são `site` (cobertura) ou `surefire-reports` (unitário).

Unitários:

Back-end:

Os testes unitários, no geral, são realizados conforme no exemplo:

```
public class Calculadora {  
  
    public int somar(int a, int b) {  
        return a + b;  
    }  
  
    public int subtrair(int a, int b) {  
        return a - b;  
    }  
  
    public int multiplicar(int a, int b) {  
        return a * b;  
    }  
  
    public double dividir(double a, double b) {
```

```
        if (b == 0) {
            throw new IllegalArgumentException("Divisão por zero não é permitida.");
        }
        return a / b;
    }
}
```

```
import org.junit.jupiter.api.Assertions;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;
//importar também a classe Calculadora;
```

```
class CalculadoraTest {
```

```
    private Calculadora calculadora;
```

```
    // A anotação @BeforeEach executa este método antes de cada teste
    @BeforeEach
```

```
    void setup() {
        calculadora = new Calculadora();
    }
```

```
    @Test
```

```
    @DisplayName("Teste de soma com números positivos")
```

```
    void testSomarComNumerosPositivos() {
```

```
        // Arrange (Preparar) - 0 que precisa ser configurado para o teste
        int a = 5;
        int b = 3;
        int resultadoEsperado = 8;
```

```
        // Act (Agir) - A ação que você quer testar
        int resultadoReal = calculadora.somar(a, b);
```

```
        // Assert (Verificar) - A verificação do resultado
        Assertions.assertEquals(resultadoEsperado, resultadoReal);
```

```
    }
```

```
    @Test
```

```
    @DisplayName("Teste de subtração com números negativos")
```

```
    void testSubtrairComNumerosNegativos() {
```

```
        // Neste exemplo, usamos a mesma estrutura
        int a = -10;
        int b = -5;
        int resultadoEsperado = -5;
```

```
        int resultadoReal = calculadora.subtrair(a, b);
```

```
        Assertions.assertEquals(resultadoEsperado, resultadoReal);
```



```

}

@Test
@DisplayName("Teste de multiplicação por zero")
void testMultiplicarPorZero() {
    // A lógica do teste é a mesma
    int a = 7;
    int b = 0;
    int resultadoEsperado = 0;

    int resultadoReal = calculadora.multiplicar(a, b);

    Assertions.assertEquals(resultadoEsperado, resultadoReal);
}

@Test
@DisplayName("Teste de divisão por um número diferente de zero")
void testDividirComNumeroDiferenteDeZero() {
    double a = 10.0;
    double b = 2.0;
    double resultadoEsperado = 5.0;

    double resultadoReal = calculadora.dividir(a, b);

    Assertions.assertEquals(resultadoEsperado, resultadoReal, "A
divisão de 10 por 2 deveria ser 5.0");
}

@Test
@DisplayName("Teste para verificar se a exceção é lançada na divisão
por zero")
void testDividirPorZeroLancaExcecao() {
    double a = 10.0;
    double b = 0.0;

    // Esta anotação verifica se a exceção correta é lançada
    Assertions.assertThrows(IllegalArgumentException.class, () -> {
        calculadora.dividir(a, b);
    });
}
}

```

 **Os métodos assert validam a saída do teste com um resultado esperado**

 **Para rodar testes unitários, use make maven-test**

Mas, caso queira rodar apenas alguns testes específicos, temos alguns comandos:

- `docker compose exec backend mvn -Dtest=MinhaClasseDeTeste test`
 - Roda testes dentro de uma classe específica
- `docker compose exec backend mvn -Dtest=MinhaClasseDeTeste#nomeDoMetodo test`
 - Roda teste de método específico dentro da classe
- `docker compose exec backend mvn -Dtest=UsuarioServiceTest,PedidoServiceTest test`
 - Roda as classes especificadas
- `docker compose exec backend mvn -Dtest=UsuarioServiceTest#deveSalvarUsuario+deveAtualizarUsuario test`
 - Múltiplos métodos na mesma classe
- É possível misturar esses comandos e usar wildcards (`*`) nesses comandos

Front-end:

A estrutura de testes segue um padrão similar ao apresentado nos modelos abaixo:

```
import { describe, it, expect } from "vitest"

describe("Grupo de testes", () => {
  it("deve fazer algo", () => {
    expect(2 + 3).toBe(5)
  })
})
```

Sobre o código:

- *describe* -> "descreve" um grupo de testes;
- *it ou test* -> define um teste individual;
- *expect* -> testa o resultado com base em um resultado esperado (análogo ao assert no JUNIT)

Palavra-chave	Uso
<code>it.only</code> / <code>test.only</code>	Roda somente esse teste
<code>it.skip</code> / <code>test.skip</code>	Pula esse teste
<code>it.todo</code>	Marca um teste para implementar depois
<code>it.concurrent</code>	Roda em paralelo com outros testes

Hook	Quando roda
<code>beforeAll</code>	1x antes de todos os testes
<code>afterAll</code>	1x depois de todos os testes
<code>beforeEach</code>	antes de cada teste
<code>afterEach</code>	depois de cada teste

Matcher	O que verifica
<code>toBe(value)</code>	Igualdade estrita
<code>toEqual(value)</code>	Igualdade profunda (objetos/arrays)
<code>toBeTruthy()</code> / <code>toBeFalsy()</code>	Valores booleanos
<code>toBeNull()</code> / <code>toBeUndefined()</code> / <code>toBeDefined()</code>	Nulos/indefinidos
<code>toContain(item)</code>	Item em array ou string
<code>toHaveLength(n)</code>	Tamanho de array/string
<code>toBeGreaterThan(n)</code> / <code>toBeLessThan(n)</code>	Comparações numéricas
<code>toThrow()</code>	Se uma função lança erro

❗ Para executar testes no front, execute **make test-front**

Para testar em arquivo específico use: `docker compose exec frontend npx vitest run caminhoArquivoTeste.jsx`

Cobertura:

Back-end:

Usa-se o comando **make maven-jacoco** para gerar o log em `target/site`

🔗 Execute o arquivo html para ver os logs no navegador

Front-end:

Use o comando **make test-front-cobertura** para produzir o log que pode ser exibido no terminal, mas também gerado na pasta `coverage`.

🔗 Execute o arquivo html para ver os logs no navegador

Mocks:

Mocks simulam um componente, dependência de um método/serviço para permitir que testes sejam executados na falta destes. Exemplo prático, suponha que não tenhamos acesso ao banco de dados remoto da faculdade, o que fazer? Usar um mock no back-end para que o back-end imagine que tenha um banco rodando de fato para então executar como deveria, caso o banco estivesse conectado.

Back-end:

Principal tecnologia para mocks é o Mockito. O exemplo abaixo mostra um cenário de uso de mock.

```
import static org.mockito.Mockito.*;
import static org.junit.jupiter.api.Assertions.*;

import org.junit.jupiter.api.Test;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.MockitoAnnotations;

class UsuarioServiceTest {

    @Mock
    private UsuarioRepository usuarioRepository; // Mock do repositório

    @InjectMocks
    private UsuarioService usuarioService; // Serviço que será testado

    public UsuarioServiceTest() {
        MockitoAnnotations.openMocks(this); // Inicializa os mocks
    }

    @Test
    void deveRetornarUsuario() {
        Usuario usuario = new Usuario(1L, "Gustavo");

        when(usuarioRepository.findById(1L)).thenReturn(Optional.of(usuario));

        Usuario resultado = usuarioService.buscarPorId(1L);

        assertEquals("Gustavo", resultado.getNome());
    }
}
```

- @Mock -> cria o Mock do banco de dados
- @InjectMocks -> insere no serviço o Mock criado

Front-end:

Muito parecido com o Mockito. Veja o exemplo de Mock para componente.

```
// Botao.test.jsx
import { render, fireEvent } from "@testing-library/react"
import { vi } from "vitest"
import Botao from "../Botao"

test("chama função ao clicar", () => {
  const mockFn = vi.fn()//Função falsa que só verifica se foi chamada
  const { getByText } = render(<Botao onClick={mockFn} />)

  fireEvent.click(getByText("Clique"))
  expect(mockFn).toHaveBeenCalledTimes(1)
})
```

Bom saber:

Por fim, para explorar mais funcionalidades das tecnologias do back-end e do front-end, algumas recomendações de itens para estudo:

- Funções anônimas (arrow function no javascript e lambda no java);
- Hoisting (javascript);
- Biblioteca Optional (java);
- Comandos de terminal linux;
- List, Map, Spread, Hash, Tables (java e javascript);
- Notações (java);
- Hooks (react);
- json;
- objetos (javascript);